

Project Report

When starting this assignment, we had to determine what are the major problems in the original code so we can properly refactor the code. Examining the only class file we have 'Main.java' we see that everything has been shoved into one java file. This is wrong and does not support OOP style at all. We also see some minor programming errors like formatting, variable naming, excess calculations, no constants, etc. After our group had dissected the original code we broke up what we were going to do in steps. First we had to modularize the code, make sure to break down the code as much as possible in their own classes and even packages.

After breaking the code down, we need to refactor the code to make sense. This includes better naming conventions, variable constants, and more error handling. The reason we want to do these things in particular is because not only is it standard to basic OOP principles, but it will help sturdy the integrity of the code from errors and bugs, to a more smooth runnable experience. This style of code also makes it so much simpler to add to the existing base in the future if we want to add more features and functionality.

So after doing these steps we have a total of 5 packages, you can look in our package diagram for a more detailed description on how they communicate with each other, but the 5 packages are registration, app, model, repo, service, and util. Registration is the parent package that holds Main.java and the other packages. The app package holds the Demo code and the main Menu code for when the program runs. Model is where our core code lies. It holds the structure of Course, Enrollment, Registrable, Session, and Student. The repo package has many classes, mainly for the JsonRepository class that can load save files in json format, and export save files in json format. The other java classes in repo are there to get the pathing data from ConfigManager and to create a TypeReference. The service package only has one class in it, that

class RegistrationServices handles all logic for adding a course, dropping a course, adding and dropping a student, and enrolling said student into courses. Our last main package, util, handles the side logic like logging, printing to the console and converting any csv to a json file. Finally our Main.java in the registration package ties everything together to run smoothly.

```
public class Main {
    static Session session;

    public static void main(String[] args) throws IOException {
        session = new Session();

        CsvToJsonConverter.convertAll();

        Log log = new Log();

        boolean demo = args.length > 0 && "--demo".equalsIgnoreCase(args[0]);

        Demo.seedDemoData(session, demo);

        Utils.println("\n=== UCA Course Registration ===\n");

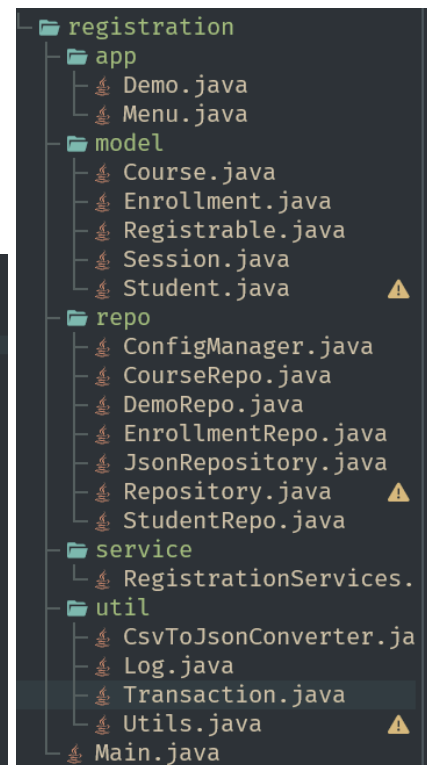
        Transaction.load(session);

        Menu.menuLoop(session);

        Transaction.save(session);

        log.write(session);

        Utils.println("Goodbye!");
    }
}
```



```

public class StudentRepo extends JsonRepository<Student> {
    public StudentRepo() {
        //super("data/students.json");
        super(ConfigManager.getPath("students"));
    }

    @Override
    protected TypeReference<Map<String, Student>> getTypeReference() {
        return new TypeReference<Map<String, Student>>() {};
    }
}

```

```

public class Demo {
    private static DemoRepo demoRepo = new DemoRepo();

    public static void seedDemoData(Session sessionObj, boolean demo) {
        if (demo) {
            demoRepo.setDemo(sessionObj);
            Utils.audit(ev:"SEED demo data", sessionObj.getAuditLog());
            Utils.println(s:"USING DEMO PARAMETERS");
        } else {
            Transaction.load(sessionObj);
        }
    }
}

```

```

public class Menu {
    public static void menuLoop(Session sessionObj) {
        Scanner sc = new Scanner(System.in);

        createSessionName(sessionObj, sc);

        while (true) {
            Utils.println(s:"\nMenu:");
            Utils.println(s:"1) Add Student");
            Utils.println(s:"2) Add Course");
            Utils.println(s:"3) Enroll student in course");
            Utils.println(s:"4) Drop student from course");
            Utils.println(s:"5) List students");
            Utils.println(s:"6) List courses");
            Utils.println(s:"7) Reload from files");
            Utils.println(s:"8) Save current sessoin");
            Utils.println(s:"0) Exit");
            Utils.println(s:"Choose: ");

            String choice = sc.nextLine().trim();

            switch (choice) {
                case "1": addStudentUI(sessionObj, sc);
                break;
            }
        }
    }
}

```

```

[estucker@DESKTOP-RUHCK7D uca-course-registration]$ java -jar target/UCA-Registration.jar
== UCA Course Registration ==
Please input your session name:
Test

Menu:
1) Add Student
2) Add Course
3) Enroll student in course
4) Drop student from course
5) List students
6) List courses
7) Reload from files
8) Save current sessoin
0) Exit
Choose:
1
Banner ID: B01284900
Name: John Doe
Email: test@cub.uca.edu

Menu:
1) Add Student
2) Add Course
3) Enroll student in course
4) Drop student from course
5) List students
6) List courses
7) Reload from files

```

```

Menu:
1) Add Student
2) Add Course
3) Enroll student in course
4) Drop student from course
5) List students
6) List courses
7) Reload from files
8) Save current sessoin
0) Exit
Choose:
5
Students:
- B01284900 John Doe <test@cub.uca.edu>

Menu:
1) Add Student
2) Add Course
3) Enroll student in course
4) Drop student from course
5) List students
6) List courses
7) Reload from files
8) Save current sessoin
0) Exit
Choose:
0
Goodbye!
[estucker@DESKTOP-RUHCK7D uca-course-registration]$

```